Run the example by pressing F5 (debugging), and you should see an output similar to listing 5.9. This example demonstrates how a semantic function can be created with SK and executed. A semantic function is the equivalent of a prompt template in prompt flow, another Microsoft tool. In this example, we define a simple prompt as a function.

It's important to note that this semantic function isn't defined as a plugin. However, the kernel can create the function as a self-contained semantic element that can be executed against an LLM. Semantic functions can be used alone or registered as plugins, as you'll see later. Let's jump to the next section, where we introduce contextual variables.

5.3.2 Semantic functions and context variables

Expanding on the previous example, we can look at adding contextual variables to the semantic function. This pattern of adding placeholders to prompt templates is one we'll review over and over. In this example, we look at a prompt template that has placeholders for subject, genre, format, and custom.

Open `SK_context_variables.py` in VS Code, as shown in the next listing. The prompt is equivalent to setting aside a `system` and `user` section of the prompt.